

Protocolo Colaborativo, Trabajos UDC

Protocolo colaborativo - unidad N°4 de DESARROLLO DE APPS

Registro de los participantes.

- **DANIEL MONTIEL SALCEDO.**
- **YEIMER OLIVERO FLOREZ.**
- **CAROLINA ISABEL PALMA DE HOYOS.**
- **JAIDER JOSÉ GARCÉS PAYARES.**

Descripción del texto o actividad a realizar.

En el presente documento realizaremos una síntesis grupal sobre los Servicios API desde el framework Flutter, donde se analizan los principales paquetes utilizados para la integración de APIs en aplicaciones móviles. Se estudian las funciones y características de herramientas como http, Dio, Chopper y GraphQL, que permiten conectar las aplicaciones con servicios externos de manera eficiente.

Además, se revisa la importancia del consumo de API dentro del desarrollo de aplicaciones con Flutter, destacando cómo estos paquetes facilitan la comunicación con servidores, el manejo de datos y la construcción de aplicaciones dinámicas y conectadas a internet.

Palabras claves.

- Servicios API
- Flutter
- Paquete http
- Dio
- Chopper
- GraphQL
- Cliente–servidor
- Consumo de datos
- JSON
- Solicitudes HTTP
- REST
- Integración

Objetivos de las lecturas o actividad a realizar.

General:

- Analizar el proceso de consumo de servicios API desde el framework Flutter, destacando el uso de paquetes como http, Dio, Chopper y GraphQL, y su importancia en la integración de aplicaciones móviles con servicios externos para el intercambio de datos de manera eficiente.

Específicos:

- Identificar las características y funciones de los paquetes http, Dio, Chopper y GraphQL utilizados para el consumo de servicios API en Flutter.

- Explicar el papel de los servicios API y la comunicación cliente–servidor en el funcionamiento de aplicaciones móviles conectadas a internet.
- Describir la importancia del consumo de datos externos y la correcta gestión de solicitudes HTTP para el desarrollo de aplicaciones dinámicas y funcionales en Flutter.

Conceptos claves y definiciones.

Servicios API: Permiten la comunicación y el intercambio de datos entre aplicaciones o sistemas.

Flutter: Framework de Google para crear apps móviles, web y de escritorio con un solo código.

Paquete http: Librería de Flutter para hacer solicitudes HTTP a APIs.

Dio: Librería avanzada para manejar peticiones HTTP con más opciones que [http](#).

Chopper: Paquete que facilita el consumo de APIs REST usando código generado automáticamente.

GraphQL: Lenguaje de consultas para APIs que permite pedir solo los datos necesarios.

Cliente–servidor: Modelo donde una app (cliente) pide información y un servidor la responde.

Consumo de datos: Proceso de obtener y usar información desde una API.

JSON: Formato de texto usado para enviar y recibir datos entre cliente y servidor.

Solicitudes HTTP: Peticiones que hace una app a un servidor (GET, POST, PUT, DELETE).

REST: Estilo de arquitectura para crear APIs basadas en solicitudes HTTP.

Integración: Conexión de una app con servicios o sistemas externos para compartir datos.

Resumen de las discusiones grupales.

CONSUMO DE APIs CON FLUTTER

El consumo de API es un aspecto importante en el desarrollo de aplicaciones móviles con flutter, ya que actúan como intermediarios en la comunicación de la aplicación con el backend y la base de datos para crear, actualizar y eliminar datos. Sin una integración de API, las aplicaciones estarían limitadas a elementos estéticos almacenados localmente, lo cual no es práctico, ya que las aplicaciones modernas requieren comunicación constante con servidores backend para acceder a información en tiempo real, permitiendo que los usuarios interactúen con datos actualizados y consistentes.

Por otro lado hablemos de REST, que se ha convertido en el estándar para diseñar servicios web distribuidos. Una API REST utiliza el protocolo HTTP como su base de comunicación, aprovechando los métodos HTTP estándar para representar operaciones sobre recursos, ya que no es un protocolo en sí, sino un estilo arquitectónico que propone principios fundamentales para crear APIs escalables, mantenibles y fáciles de entender.

El primero de estos principios es la independencia de estado, lo que significa que cada solicitud HTTP contiene toda la información necesaria para procesarla sin que el servidor necesite recordar el contexto de solicitudes anteriores. Otro principio clave es la separación del cliente con el servidor, permitiendo que cada uno evolucione por su lado y luego se realice la respectiva conexión.

El paquete http de Dart es la solución más elemental y directa para realizar solicitudes HTTP en Flutter, ya que proporciona una API basada en Futures que se integra perfectamente con el modelo async y await nativo de dart. Y aunque es simple, el http tiene también sus limitaciones por ello no es lo más adecuado cuando trabajamos con aplicaciones complejas o que pueden ser escalables, ejemplo de ello es que no proporciona mecanismos integrados para interceptores, que son necesario al momento de agregar headers, como tokens para autenticación de las solicitudes.

Consumo de datos y solicitudes HTTP.

El consumo de datos mediante solicitudes HTTP consiste en la interacción de una aplicación cliente con un servidor para obtener, enviar o modificar información. En Flutter esta comunicación se realiza principalmente a través de paquetes como http, Dio o Chopper, que facilitan la realización de solicitudes GET, POST, PUT o DELETE hacia APIs REST o GraphQL.

Estas solicitudes permiten que la aplicación reciba datos en formatos estándar como JSON o XML, los procese y los muestre dinámicamente en la interfaz de usuario. Además los paquetes de Flutter proporcionan herramientas para manejar errores, configurar tiempos de espera, gestionar headers y caché de respuestas, lo que asegura que la aplicación funcione de manera eficiente y confiable.

El consumo de datos mediante HTTP es esencial para aplicaciones modernas que requieren actualización en tiempo real, sincronización con servicios en la nube y comunicación constante con servidores. Su correcta implementación garantiza que la aplicación pueda interactuar con servicios externos de forma segura, optimizando el rendimiento y la experiencia del usuario final.

Uso de paquetes específicos: http, Dio, Chopper y GraphQL.

En Flutter, el consumo de servicios API es esencial para conectar las aplicaciones con servidores y bases de datos externas, y esto permite el intercambio de información en tiempo real. Para lograrlo, existen distintos paquetes que facilitan la comunicación mediante solicitudes HTTP, entre los más principales tenemos: http, Dio, Chopper y GraphQL. Cada uno ofrece herramientas específicas que ayudan a optimizar la conexión entre cliente y servidor, mejorar la gestión de datos y construir aplicaciones más dinámicas y funcionales.

El paquete http: Es la opción más básica y sencilla ya que permite realizar solicitudes como GET, POST, PUT y DELETE, y maneja datos en formato JSON de manera directa. Sin embargo, al ser tan simple, esta opción se ve limitada en aplicaciones más complejas, ya que no cuenta con funciones integradas para interceptores, manejo avanzado de errores o autenticación, lo que puede dificultar su uso en entornos de producción.

Dio: Es un cliente HTTP más completo que incluye características avanzadas como interceptores, manejo de errores, autenticación mediante tokens, cancelación de

solicitudes y control de tiempos de espera. Todas estas funciones lo convierten en una herramienta con bastantes capacidades y segura para aplicaciones de mediana y gran escala.

Chopper: Destaca por su capacidad para generar código automáticamente y estructurar mejor la comunicación con APIs REST, lo cual favorece la mantenibilidad y la organización del proyecto.

GraphQL: Se caracteriza por tener un enfoque diferente al uso de APIs, ya que permite realizar consultas precisas solicitando solo los datos necesarios, lo que le permite reducir el tráfico de red y mejorar la eficiencia. Este método es ideal para aplicarlo en aplicaciones que manejan grandes volúmenes de información o requieren actualizaciones en tiempo real.

Comunicación cliente–servidor y gestión de datos en aplicaciones móviles.

La comunicación cliente–servidor en aplicaciones móviles es el proceso mediante el cual una app (cliente) se conecta con un servidor para solicitar y recibir información. El servidor responde a las peticiones del cliente enviando datos, generalmente en formato JSON, que luego son procesados y mostrados en la interfaz de usuario.

Esta interacción permite que las aplicaciones sean dinámicas y estén actualizadas en tiempo real, la gestión de datos implica cómo la app maneja, almacena y sincroniza esa información, garantizando eficiencia, seguridad y buen rendimiento.

En conjunto, una correcta comunicación cliente–servidor y una buena gestión de datos son fundamentales para crear aplicaciones móviles funcionales, rápidas y confiables.

Encuentros conceptuales.

Durante las discusiones grupales, coincidimos en que el uso de servicios API es esencial para conectar las aplicaciones Flutter con fuentes de datos externas, se reconoció la importancia de comprender la estructura cliente–servidor y de manejar correctamente formatos de intercambio como JSON.

También se comprendió que los paquetes http, Dio, Chopper y GraphQL ofrecen herramientas útiles y complementarias para realizar solicitudes HTTP, manejar respuestas, interceptores y optimizar la comunicación entre la aplicación y el servidor.

Desencuentros conceptuales.

Al inicio del trabajo, existieron pequeñas diferencias en la comprensión del funcionamiento interno de los paquetes Dio y Chopper, especialmente en la forma en que ambos gestionan las solicitudes HTTP y la generación automática de código. Sin embargo, tras la revisión conjunta de documentación y ejemplos prácticos, el grupo logró unificar los criterios y comprender las ventajas y limitaciones de cada uno.

Metodología de trabajo (Cómo se hizo la actividad colaborativa).

La actividad colaborativa se desarrolló mediante reuniones virtuales realizadas a través de Google Meet, donde cada uno de los integrantes aportó ideas y conocimientos sobre el consumo de servicios API en Flutter y el uso de paquetes como http, Dio, Chopper y GraphQL. Luego en conjunto se compartió la información recopilada y se debatieron los

puntos más relevantes para elaborar una síntesis grupal coherente y completa, por último realizamos cada uno de los apartados de este protocolo.

Conclusiones.

El estudio y análisis del consumo de servicios API en Flutter permitió comprender la relevancia de la integración entre cliente y servidor en las aplicaciones modernas, los paquetes **http**, **Dio**, **Chopper** y **GraphQL** ofrecen distintas alternativas para realizar solicitudes HTTP y manejar datos externos de manera eficiente.

Se concluye que el dominio de estas herramientas es fundamental para los desarrolladores de software, ya que permite crear aplicaciones móviles conectadas, dinámicas y adaptadas a las necesidades actuales del entorno tecnológico, se fortalecieron las habilidades colaborativas y la comprensión grupal sobre la arquitectura y comunicación de servicios en Flutter.

Discusiones y recomendaciones.

Se recomienda emplear http para proyectos pequeños o educativos, mientras que **Dio** es más apropiado para aplicaciones de mayor complejidad debido a sus funcionalidades avanzadas.

Se sugiere profundizar en el uso de **Chopper** y **GraphQL**, ya que ofrecen estructuras más organizadas y flexibles para la gestión de datos

Referencias Bibliográficas.

- Alessandria, S., & Kayfitz, B. (2021). Flutter Cookbook: Over 100 proven techniques and solutions for app development with Flutter 2.2 and Dart. Packt Publishing.
- Smyth, N. (2020). Android Studio 4.0 Development Essentials – Kotlin Edition: Build Android Apps with Android Studio and Kotlin. Packt Publishing. (Aunque se centra en Kotlin/Android, contiene buen contexto sobre consumo de APIs y puede aplicarse conceptualmente a Flutter).
- “Using GraphQL with Flutter: A tutorial with examples.” LogRocket Blog. (2021). Recuperado de <https://blog.logrocket.com/using-graphql-with-flutter-a-tutorial-with-examples/>
- “Simplify API Services in Flutter with chopper.” Medium – PubDev Essentials. (2025, Agosto 11). Recuperado de <https://medium.com/pubdev-essentials/simplify-api-services-in-flutter-with-chopper-1552dd84f2f6>

- Tsuji, K. (2023). Democratizing application development with AppSheet: A citizen developer's guide to building rapid low-code applications with powerful features of AppSheet. Packt Publishing. Disponible en:

https://unicartagena.elogim.com/authmeta/login.php?url=https://ebSCO.unicartagena.aprox.elogim.com/login.aspx?direct=true&db=eds_vle&AN=eds_vle.AH40988833&lang=es&site=eds-live

Komal Pardeshi. (2025, June 3). Flutter Dio Tutorial: The Ultimate HTTP Client for Flutter Development. Mobisoft Infotech.

<https://mobisoftinfotech.com/resources/blog/flutter-development/flutter-dio-tutorial-http-client>